



Week 2 Lecture 2

▼ Class	BSCCS2003
🕒 Created	@September 14, 2021 4:50 PM
🔗 Materials	
# Module #	11
▼ Type	Lecture
☰ Week #	2

Efficiency of encoding

Efficiency

- What is the most common language on the web?
 - As of now, it is English
- Should all the characters be represented with the same number of bits?
- Example:
 - A text document with 1,000 words, which equates to approximately 5,000 characters (including spaces)
 - According to UCS-4 encoding
$$32 \text{ bits} \times 5,000 = 160,000 \text{ bits}$$
 - According to ASCII encoding
$$8 \text{ bits} \times 5000 = 40,000 \text{ bits}$$
 - According to the original 7-bit ASCII which was sufficient for English
$$7 \text{ bits} \times 5000 = 35,000 \text{ bits}$$
 - Minimum needed to encode just 'a' - 'z', numbers and some special characters could fit in 6-bits: 30,000 bits
 - Optimal coding based on frequency of occurrence of a letter
 - 'e' is the most common letter, 't', 'a', 'o', ...
 - Huffman or similar encoding: ~10,000 - 20,000 bits, possibly less

Is this problem solvable in general?

- It is impossible to encode by actual character frequency: depends on the text
 - Just use compression methods like "zip" instead
- But can encoding be a good halfway point?

Example:

- Use 1-byte for most common alphabets
- Group others according to frequency, have "prefix" codes to indicate

Prefix Coding

 1st byte	 2nd byte	 3rd byte	 4th byte	 Free bits	 Max. expressible Unicode val.
<u>0</u> xxxxxxx				7	007F hex (127)
<u>110</u> xxxxx	10xxxxxx			(5+6)=11	07FF hex (2047)
<u>1110</u> xxx	10xxxxxx	10xxxxxx		(4+6+6)=16	FFFF hex (65535)
<u>11110</u> xx	10xxxxxx	10xxxxxx	10xxxxxx	(3+6+6+6)=21	10FFF hex (1,114,111)

- If the first bit starts with a 0, take the remaining 7 free bits as an ASCII code

Example

	A	ᄀ	好	丕
Code point	U+0041	U+05D0	U+597D	U+233B4
UTF-8	41	D7 90	E5 A5 BD	F0 A3 8E B4
UTF-16	00 41	05 D0	59 7D	D8 4C DF B4
UTF-32	00 00 00 41	00 00 05 D0	00 00 59 7D	00 02 33 B4

Source: <https://www.w3.org/International/articles/definitions-characters>

UTF-8

- Stands for Unicode Transformation Format
- Uses 8-bits for most common characters: ASCII subset
 - All ASCII documents are automatically UTF-8 compatible
- All other characters can be encoded based on prefix string
- More difficult for text processors:
 - first check prefix
 - linked list through chain of prefixes possible
 - Still more efficient for majority of documents
- Most common encoding in use today